



UNIVERSITY OF BUEA

Faculty of Engineering and Technology

Department of Computer Engineering

AI-POWERED EXPERT SYSTEM FOR CROP DISEASE DIAGNOSIS AND TREATMENT RECOMMENDATION

TECHNICAL PROJECT REPORT - TOMATO DISEASE EXPERT SYSTEM

Submitted by:

NAME	MATRICULE
DERRICK MBUNO FORCHA	FE25P117
TAKEM JIM-RAWLINGS	FE25P106

Module: Final Practical Project - Intelligent Agricultural Expert System

Repository: github.com/JIM440/cef-610-expert-system

ABSTRACT

This report presents the design, implementation and testing of a Crop Disease Expert System for tomato disease diagnosis and treatment recommendation. The system assists farmers and agricultural experts by accepting selected symptoms and applying a database-driven rule engine to determine likely diseases and recommended treatments. It also provides image recognition support for uploaded plant images, stores diagnosis history, and allows PDF report export. The implementation uses Streamlit for the user interface and PostgreSQL for the knowledge base and consultation records. The system is designed to be transparent, maintainable and suitable for academic demonstration, with expert rules stored in the database rather than hardcoded in the interface.

TABLE OF CONTENTS

TABLE OF CONTENTS 2

1. INTRODUCTION AND PROJECT BRIEF ALIGNMENT 4

2. SYSTEM OVERVIEW 4

 2.1 User Roles and Capabilities 5

 2.2 Sample Credentials 5

 2.3. Environment Configuration 5

 2.4. Installation and Running the System 5

3. SYSTEM ARCHITECTURE 6

 3.1 Application Layers 6

 3.2 Project File Structure 7

4. SYSTEM ANALYSIS AND DESIGN 7

 4.1 Rule-Based Inference Flowchart 7

5. DATABASE DESIGN 8

 5.1 Final Table Inventory 9

 5.2 Key Table Descriptions 9

6. EXPERT-SYSTEM REASONING 10

 6.1 Knowledge Acquisition 10

 6.2 Rule-Based Inference Engine 11

 6.3 Hybrid Expert System 11

7. IMAGE RECOGNITION MODULE 12

 7.1 Image Workflow 12

8. USER INTERFACE	13
8.1 Farmer Pages	13
8.2 Expert/Admin Pages	15
9. KNOWN LIMITATIONS, SAFEGUARDS AND FUTURE WORK.....	17
9.1 Limitations	17
9.2 Future Improvements.....	17
10. CONCLUSION	18

LIST OF FIGURES

Figure 1: Complete system overview showing users, expert-system process, database and outputs.....	4
Figure 2: Rule-based inference flowchart.	8
Figure 3: Farmer Diagnosis Interface - Symptom Selection and Disease Prediction.....	12
Figure 4: Diagnosis Results for Farmers Page Showing the rule based results, AI prediction and proposed treatment	13
Figure 5: Farmer Consultation History - Rule-Based and AI Diagnosis Records	14
Figure 6: Image Recognition Module - AI-Powered Symptom Extraction and Diagnosis	14
Figure 7: Dashboard Page where we can view recent consultations and a graph of most frequent diseases predicted	16
Figure 8: Expert Rule Management - Here we di define the rule and associate the disease to symptoms and proposed treatments	17

1. INTRODUCTION AND PROJECT BRIEF ALIGNMENT

This report documents the design, implementation, and testing of a Tomato Disease Expert System built to satisfy the Final Practical Project brief: "AI-Powered Expert System for Crop Disease Diagnosis and Treatment Recommendation." The system combines knowledge-based expert-rule reasoning with a PostgreSQL-backed knowledge base and a Streamlit user interface. Image-based diagnosis support is included to allow farmers and experts to upload crop images for review or recognition workflow support.

The system targets tomato as the currently supported crop because tomato diseases are common, agriculturally significant, and well documented. The design can be expanded to additional crops by adding new disease, symptom, treatment and rule data.

2. SYSTEM OVERVIEW

The system assists farmers and agricultural experts in diagnosing crop diseases from manually selected symptoms and image-related evidence. A farmer can run self-diagnosis, view personal history and export reports. An expert or administrator can manage the knowledge base, create rules, diagnose generally or for a farmer, and review reports.

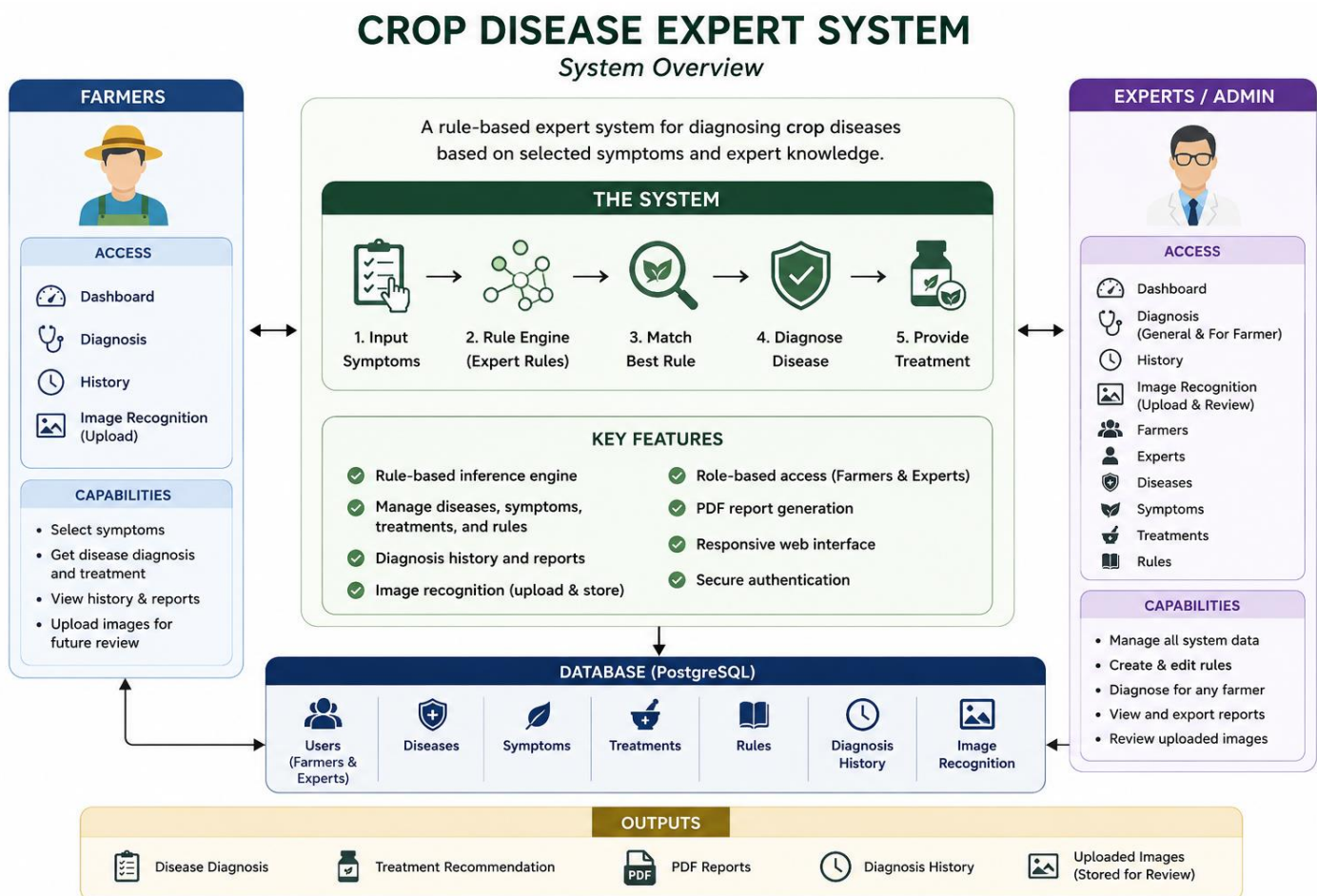


Figure 1: Complete system overview showing users, expert-system process, database and outputs.

2.1 User Roles and Capabilities

Farmer

- Log in, view farmer dashboard, perform self-diagnosis
- Upload images for image recognition workflow
- View diagnosis history, export diagnosis reports as PDF

Expert / Administrator

- Log in, diagnose for himself or for a registered farmer
- Manage diseases, symptoms, treatments and rules
- Manage farmer and expert records
- Review uploaded images
- Search, filter and export diagnosis history

2.2 Sample Credentials

Username	Password	Role	Access Level
Expert	expert123	EXPERT	Expert workspace and knowledge-base management
farmer1	farmer123	FARMER	Personal diagnosis, history and reports

2.3. Environment Configuration

Before running the system, create a .env file from the provided .env.example template. The required values are (refer to the .env.example in codebase for actual credentials):

```
DB_HOST=localhost
DB_PORT=5433
DB_NAME=crop_expert_system
DB_USER=postgres
DB_PASSWORD=<your_postgres_password>
GEMINI_API_KEY=<your_gemini_api_key>
GEMINI_MODEL=gemini-2.0-flash
```

2.4. Installation and Running the System

Prerequisites

- Python 3.10 or newer
- PostgreSQL
- Streamlit

- Required Python packages from requirements.txt
- A configured .env file containing database credentials

Setup Steps

- Clone or open the project folder.
- Create and activate a Python virtual environment.
- Install dependencies using `pip install -r requirements.txt`.
- Create the PostgreSQL database.
- Run `database/schema.sql` and `database/seed.sql`.
- Configure `.env`.
- Start the application using `python run.py` or `python -m streamlit run app/ui/streamlit_app.py`.

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
createdb crop_expert_system
psql -U postgres -d crop_expert_system -f database/schema.sql
psql -U postgres -d crop_expert_system -f database/seed.sql
python run.py
```

3. SYSTEM ARCHITECTURE

The application follows a layered architecture. The Streamlit user interface communicates with service classes that coordinate diagnosis, image recognition, history and report generation. The service layer calls the rule engine and repositories, while PostgreSQL stores all user, knowledge-base, rule and diagnosis-history records.

3.1 Application Layers

Layer	Responsibility
User Interface	Streamlit pages, navigation, forms, cards, filters, history and report export
Service Layer	Coordinates diagnosis, image recognition, consultation history and PDF workflow
Expert System Layer	Loads rules, compares selected symptoms, determines disease and treatment
Repository Layer	Executes SQL queries and maps database results to application objects
Database Layer	Stores farmers, experts, diseases, symptoms, treatments, rules, images and diagnosis history

3.2 Project File Structure

```
app/  
  ai/          Gemini image symptom extraction  
  expert_system/  Rule construction, matching, confidence, explanations  
  repositories/   PostgreSQL data access  
  services/      Diagnosis, image, consultation, PDF workflows  
  ui/  
    components/   Shared Streamlit views and forms  
    pages/admin/   Expert-capable pages  
    pages/farmer/  Farmer pages  
    streamlit_app.py Authentication and navigation entry point  
  config.py      Environment-backed configuration  
  database.py    Connections, query helpers, schema health checks  
  
database/  
  schema.sql     Canonical 15-table PostgreSQL schema  
  seed.sql       Current knowledge base and demo data  
  consolidate_29_to_15.sql One-time legacy schema migration  
  add_naive_bayes_columns.sql Additive Phase 6/7 columns  
  
run.py          Application launcher  
requirements.txt Python dependencies
```

4. SYSTEM ANALYSIS AND DESIGN

4.1 Rule-Based Inference Flowchart

The diagnosis process begins when a user selects symptoms. The system loads active rules, compares each rule against the selected symptoms, selects the best matching rule and returns the disease, explanation and treatment recommendation. If no exact match exists, the system can suggest the closest possible match based on partial evidence.

RULE-BASED INFERENCE FLOWCHART

Crop Disease Expert System

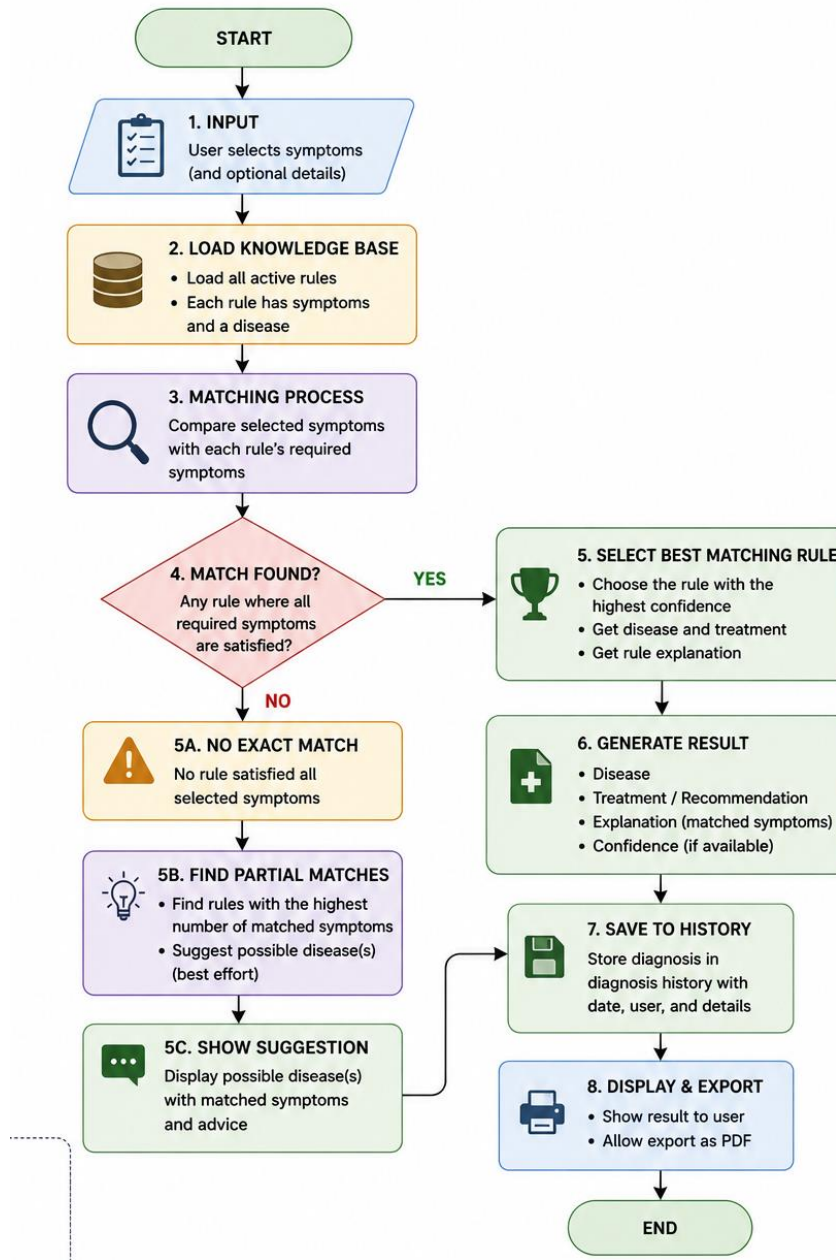


Figure 2: Rule-based inference flowchart.

5. DATABASE DESIGN

The database stores user accounts, diseases, symptoms, treatments, rules, diagnosis records and uploaded-image records. Many-to-many relationships such as disease-to-symptom and rule-to-symptom are represented using junction tables.

5.1 Final Table Inventory

Group	Tables
Users	app_user
Knowledge base	crop, disease, symptom, treatment, environmental_factor, disease_symptom
Expert rules	rule, rule_symptom, rule_environment, rule_treatment
Consultations	consultation, consultation_symptom, consultation_environment, consultation_treatment

5.2 Key Table Descriptions

Table	Purpose
app_user	Login credentials, role (ADMIN/EXPERT/FARMER), status, name, and farmer contact/profile fields in a single consolidated table
Crop	Supported crop records; currently tomato only
Disease	Tomato diseases with description and explanation template used by rule-based diagnosis results
Symptom	Reusable observable symptom definitions shared across disease facts, rules, and consultation evidence
Treatment	Reusable treatment recommendations linked to expert rules and stored per consultation
environmental_factor	Normalised category/value pairs e.g. Humidity:High, Temperature:Warm, Plant spacing:Dense
disease_symptom	Descriptive disease-to-symptom knowledge; weight column drives Naive Bayes priors
Rule	Core expert rule: one disease, a name, a confidence score, and an active flag

Table	Purpose
rule_symptom	Symptoms required by each rule (junction table)
rule_environment	Environmental factors required by each rule (junction table)
rule_treatment	Prioritised treatments produced when a rule fires (junction table)
Consultation	One diagnosis event: both rule-based and Naive Bayes results, source, type, match tier, and Gemini trace
consultation_symptom	Every symptom submitted per consultation with match flag
consultation_environment	Every environmental factor submitted per consultation with match flag
consultation_treatment	Treatments recommended for a consultation by the matched rule

The database was consolidated through a documented, deliberate normalisation process. The final schema retains full referential integrity, avoids arrays/CSV/JSON values in any column, and represents every many-to-many relationship with a proper junction table.

6. EXPERT-SYSTEM REASONING

6.1 Knowledge Acquisition

The knowledge base was compiled from agricultural literature on tomato diseases. Eight diseases are currently encoded in the database (but can be extended or added by the expert):

Disease	Example symptoms / context
Early Blight	Brown spots, yellow leaves, concentric rings, lower leaves affected.
Late Blight	Dark water-soaked lesions, white mould, rapid spread under wet conditions.
Septoria Leaf Spot	Small circular spots, dark borders, grey centres.
Leaf Mould	Yellow patches and mould on lower leaf surfaces.
Bacterial Spot	Small dark water-soaked spots affecting leaves and fruit.
Tomato Mosaic Virus	Mosaic mottling, leaf distortion and stunted growth.

Fusarium Wilt	Yellowing lower leaves, wilting and vascular discolouration.
Healthy	No major visible disease symptoms.

Twelve symptoms, 15 environmental factors, and 10 treatments are defined, with 13 expert rules and 23 disease–symptom knowledge associations stored in the database.

6.2 Rule-Based Inference Engine

The inference engine performs forward chaining over the active rule set. For each active rule, it computes:

$$\text{match ratio} = \text{matched required facts} / \text{total required facts}$$

$$\text{matched score} = \text{int}(\text{match ratio} \times \text{rule confidence score})$$

- A complete match is assigned tier HIGH.
- A partial match at or above 50% is assigned tier LOW.
- If no rule reaches 50%, the engine still returns the best non-zero partial match as LOW rather than refusing a result.
- Rules with no matched evidence at all are not candidates.
- Results are ranked by tier then matched score; the top-ranked rule becomes the stored matched_rule_id.

The system stores the selected disease, final confidence, match tier, matched rule, matched symptoms, matched environmental factors, explanation text, and recommended treatments — making every diagnosis fully traceable to specific database evidence rather than an unexplained output.

Example rule (Early Blight):

IF Brown spots AND Yellow leaves AND High humidity
THEN Early Blight (confidence 85%)

6.3 Hybrid Expert System

Every manual or image-assisted diagnosis sends the identical symptom and environmental-factor evidence to both reasoning paths simultaneously:

- Rule-Based Result — the expert-rule inference output: matched rule, explanation, confidence, and treatments.
- AI Prediction (Naive Bayes) — the independently normalised disease probability described above.

Both results are displayed side by side in the user interface. **The system never silently merges them into a single confidence score. A separate agreement indicator states whether both methods selected the same disease. Recommended treatments remain tied exclusively to the matched expert rule — Naive Bayes never independently proposes treatments.**

Database storage: the rule result is stored in `final_disease_id`, `final_confidence`, `matched_rule_id`, and `explanation`. The AI result is stored in `ai_predicted_disease_id`, `ai_confidence`, and `ai_model_version`. If no expert rule matches but Naive Bayes returns a prediction, the consultation is retained with `match_tier = NONE` and an empty rule result.

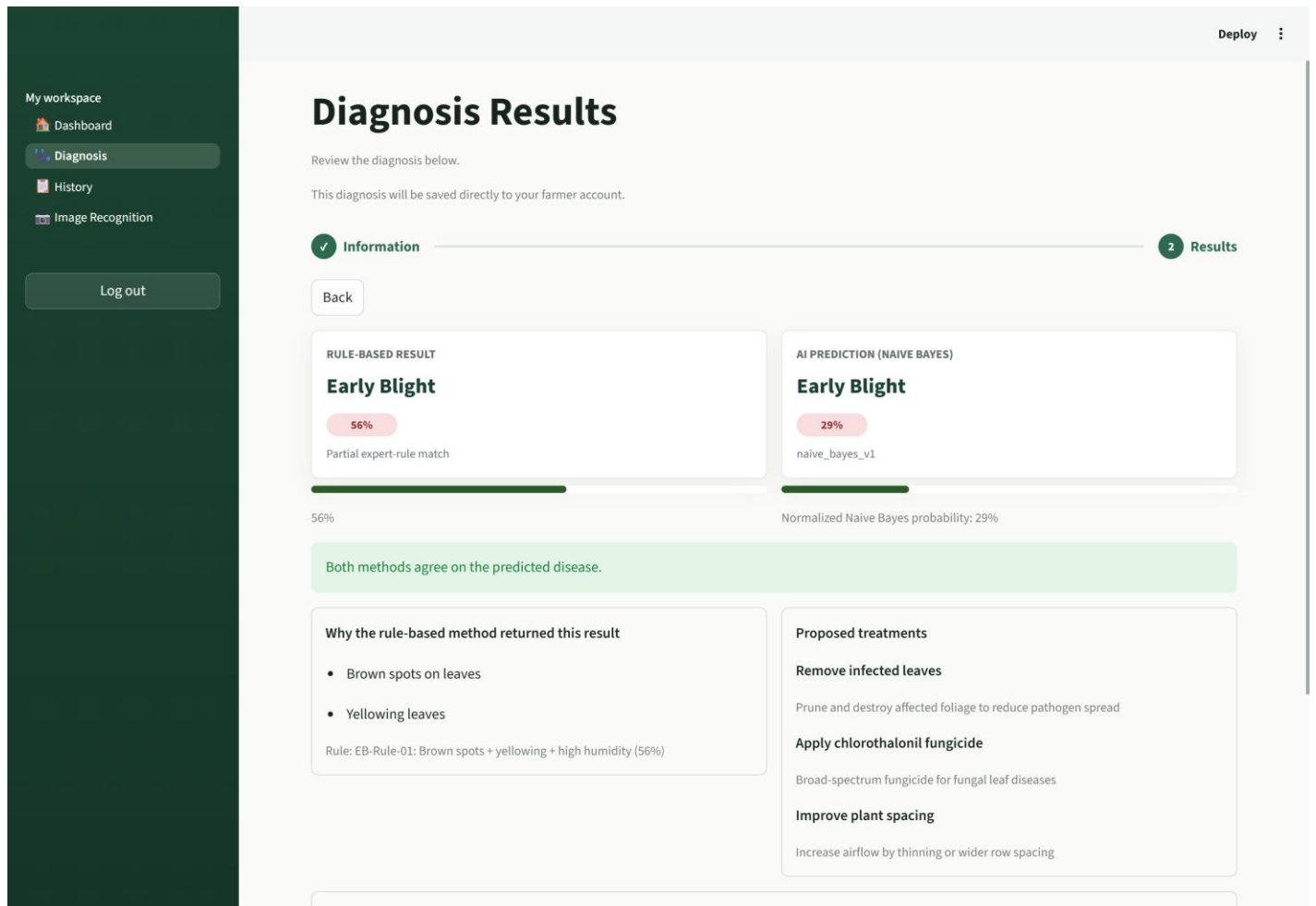


Figure 3: Farmer Diagnosis Interface - Symptom Selection and Disease Prediction

7. IMAGE RECOGNITION MODULE

The image recognition page allows users to upload crop images for image-assisted diagnosis or expert review. In the current project, the module supports the workflow required for image upload and diagnosis support while preserving the rule-based expert system as the diagnostic authority.

7.1 Image Workflow

1. User uploads a crop image.
2. The system validates and stores or processes the image according to the configured workflow.
3. Visible symptoms may be extracted or reviewed.
4. Mapped symptoms are passed to the rule engine.

5. Diagnosis result is displayed and saved to history.

8. USER INTERFACE

The interface is built with Streamlit and separates farmer pages from expert/admin pages. The design uses a clean white and green theme, card-based management pages and simple navigation.

8.1 Farmer Pages

Page	Responsibility
Dashboard	Shows personal diagnosis statistics and recent activity.
Diagnosis	Allows the farmer to select symptoms and run self-diagnosis.
History	Shows the farmer's previous diagnoses and report export option.
Image Recognition	Allows image upload for image-assisted diagnosis support.

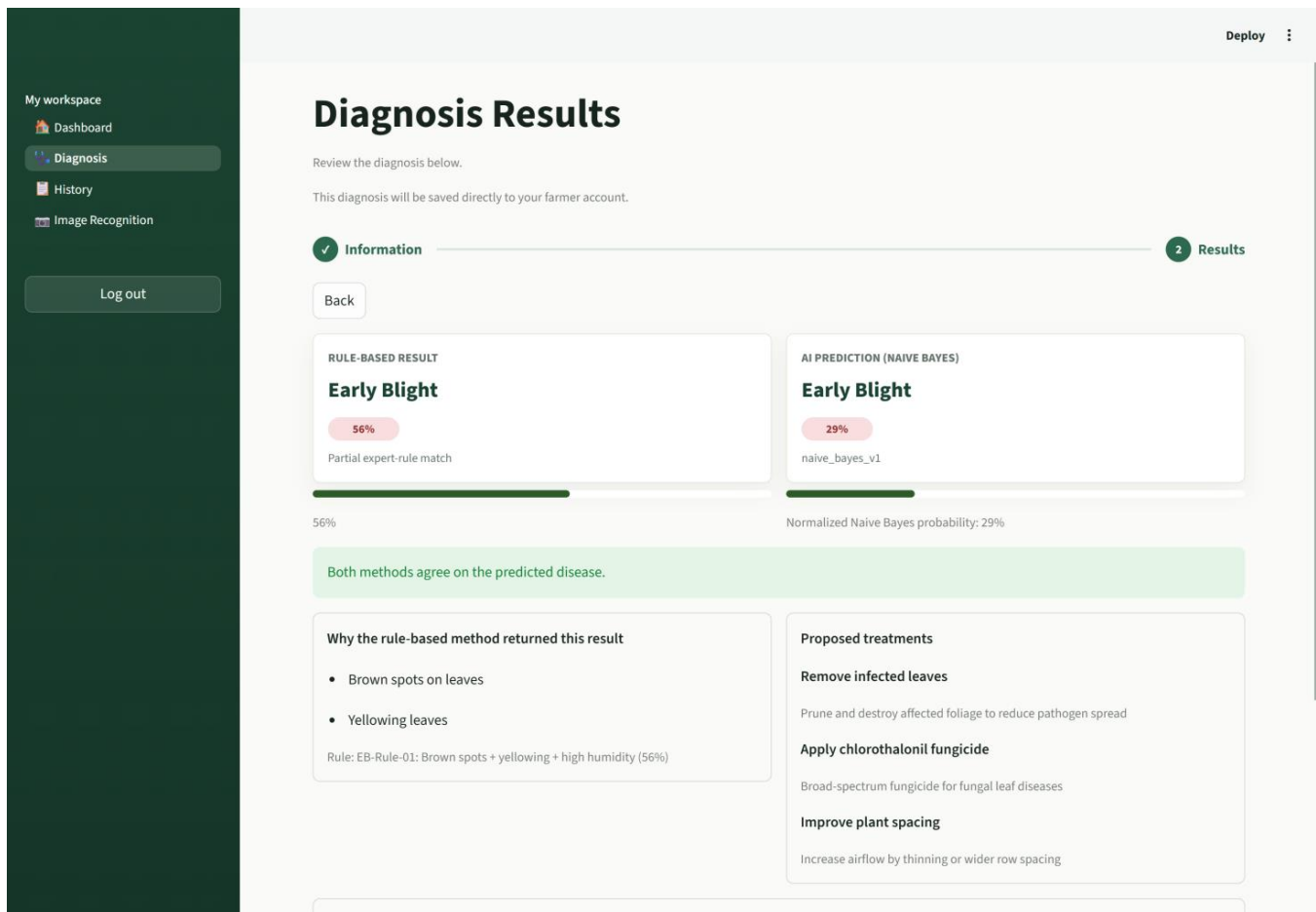


Figure 4: Diagnosis Results for Farmers Page Showing the rule based results, AI prediction and proposed treatment

My workspace

Dashboard

Diagnosis

History

Image Recognition

Log out

Deploy

My History

Review rule-based and Naive Bayes results, then edit and export the filtered consultations.

Source

☒ All diagnoses
 ☐ Manual diagnosis
 ☐ Image recognition

[Consultations](#)
[Export PDF](#)

Date	Rule-based Result	AI Prediction	Agreement	Submitted symptoms	Source
Jun 22, 2026 20:54	No result	Early Blight (26%)	Unavailable	Leaf drop	Image recognition
Jun 22, 2026 20:52	Late Blight (45%)	Powdery Mildew (25%)	Disagree	Brown spots on leaves, Dark water-soaked lesions, Leaf curling, Mosaic leaf pattern	Manual selection
Jun 22, 2026 20:49	Tomato Mosaic Virus (86%)	Tomato Mosaic Virus (57%)	Agree	Brown spots on leaves, Leaf curling, Leaf drop, Mosaic leaf pattern, Purple leaf vein	Manual selection
Jun 22, 2026 20:31	Early Blight (56%)	Early Blight (29%)	Agree	Brown spots on leaves, Leaf drop, Purple leaf veins, White powdery coating, Wilting	Manual selection
Jun 22, 2026 20:15	Tomato Mosaic Virus (43%)	Blossom End Rot (30%)	Disagree	Brown spots on leaves, Fruit rot at blossom end, Leaf curling, Leaf drop, Purple leaf	Manual selection
Jun 08, 2026 11:53	Tomato Mosaic Virus (86%)	No result	Unavailable	Mosaic leaf pattern, Stunted growth	Manual selection

Figure 5: Farmer Consultation History - Rule-Based and AI Diagnosis Records

My workspace

Dashboard

Diagnosis

History

Image Recognition

Log out

Deploy

Image Recognition

Upload a clear image of the affected plant part. Gemini extracts symptoms; the expert system rule base performs the diagnosis.

Upload plant image

Drag and drop file here

Limit 200MB per file • JPG, JPEG, PNG

Browse files

tomato-diseases-and-treatment.jpg

21.9KB

×

JPG, PNG, JPEG. Max 5MB. Images are sent to Gemini and are not stored.



Uploaded image

Analyze Image

How it works

1. Upload a clear image of the affected leaf, stem, or fruit.
2. Gemini identifies visible symptoms from the database catalog.
3. Expert rules match those symptoms to disease and treatment records.
4. The consultation stores text only: source and Gemini extraction, not the image file.

Figure 6: Image Recognition Module - AI-Powered Symptom Extraction and Diagnosis

14

8.2 Expert/Admin Pages

Page	Responsibility
Dashboard	Shows system statistics, disease frequency and recent consultations.
Diagnosis	Allows general diagnosis or diagnosis for a selected farmer.
History	Allows searching, filtering, reviewing and exporting diagnosis records.
Image Recognition	Allows upload/review of plant images.
Farmers	Allows viewing, adding and editing farmer accounts.
Experts	Allows viewing and adding expert accounts.
Diseases	Allows disease knowledge management.
Symptoms	Allows symptom catalog management.
Treatments	Allows treatment recommendation management. We can add and edit treatments
Rules	Allows creation and editing of expert rules.

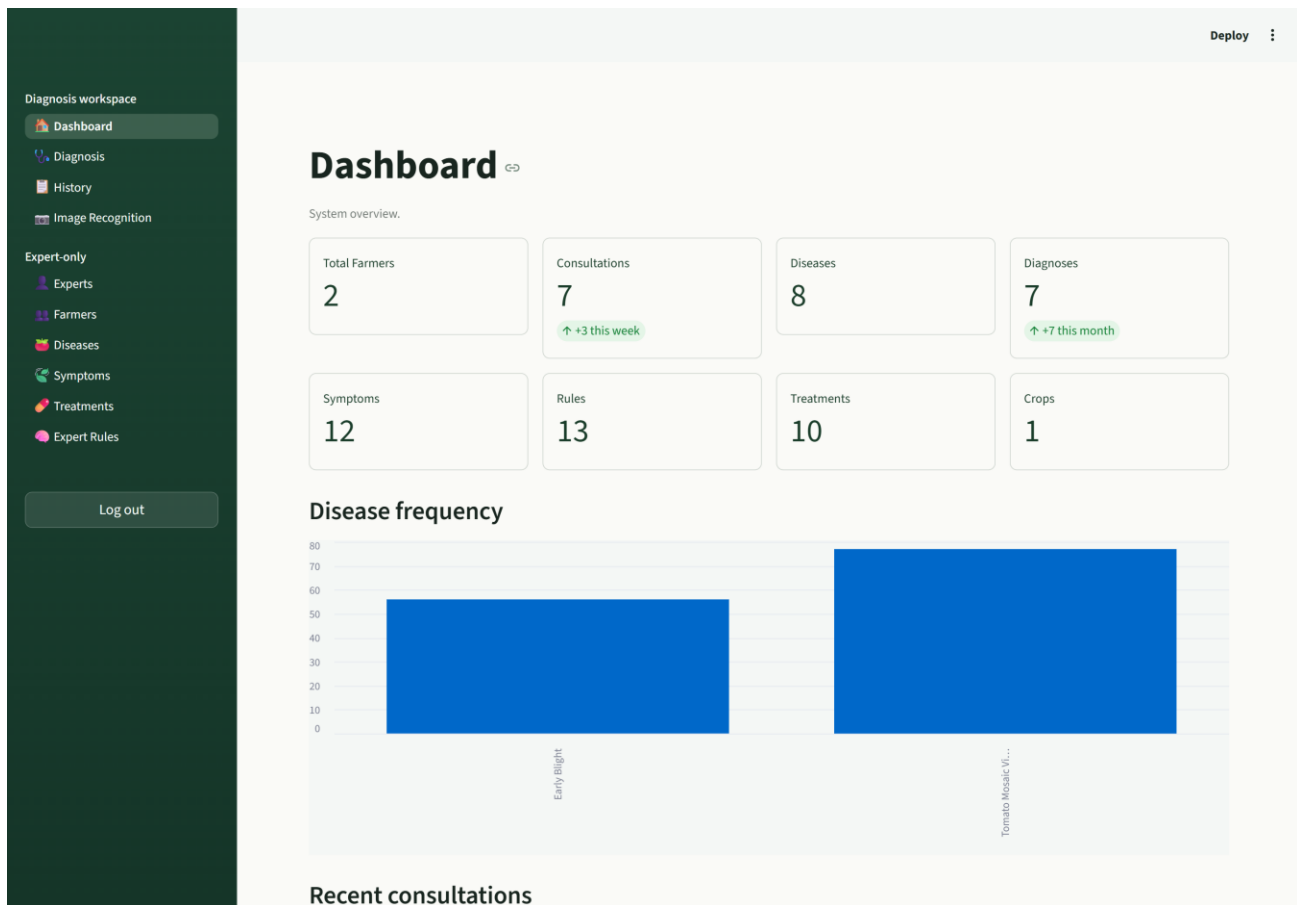


Figure 7: Dashboard Page where we can view recent consultations and a graph of most frequent diseases predicted

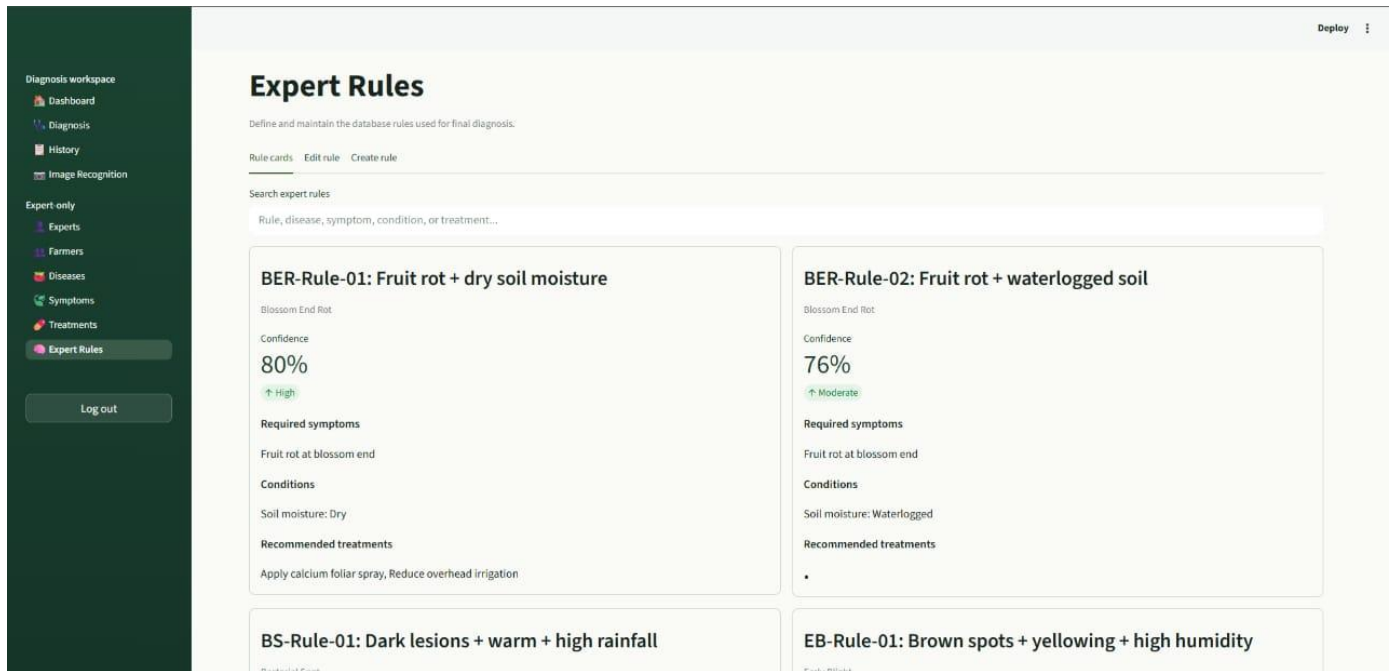


Figure 8: Expert Rule Management - Here we define the rule and associate the disease to symptoms and proposed treatments

9. KNOWN LIMITATIONS, SAFEGUARDS AND FUTURE WORK

9.1 Limitations

- The current knowledge base focuses on tomato diseases.
- Diagnosis quality depends on the accuracy of expert rules.
- Image recognition support does not replace expert agricultural judgment.
- The system is not a laboratory confirmation tool.
- Additional crops require new diseases, symptoms, treatments and rules.

9.2 Future Improvements

- Add more crops such as maize, cassava and pepper.
- Add expert approval workflow for uncertain diagnoses.
- Add multilingual symptom and treatment labels.
- Add weather-data integration for environmental evidence.

10. CONCLUSION

The Crop Disease Expert System successfully demonstrates how expert-system principles can be applied to agricultural disease diagnosis. The system allows farmers to diagnose crop diseases using selected symptoms, enables experts to manage the knowledge base and rules, stores history records, supports image upload workflows and generates reports. Its rule-based design makes the diagnosis process transparent and easy to explain during project defense. The database-centered architecture also makes the system maintainable and expandable for future crops and AI-based improvements.

The complete source code, database scripts, automated tests, and this report are available at <https://github.com/JIM440/cef-610-expert-system>.